Round 1: overview

Platypus with one TEE call in place of the two zero-knowledge proofs
Serials are one-time public keys: no secret that can spend a user's funds reaches the enclave

Parties. red untrusted green runs attested code, may crash orange trusted for integrity, curious about privacy
Guarantees. bank rests on a check the bank does itself enclave key rests on the secrecy of the enclave's signing key enclave memory: privacy rests on it
blue sealed to the attested enclave

Users (Alice pays, Bob receives)
May double spend, replay old states, forge states, collude with the bank

On the device
seed: never leaves the device; derives every one-time key and blind
state $S = (b,\ pk,\ r)$: balance, serial key, blind
bank signature σ on $C = \mathrm{Com}(b, pk, r)$: makes it money
one-time keys sk_i, pk_i: pk_i is the serial of the i-th state

To pay or receive
each builds its new state C' with its next key and blind
both sign the transaction $m = H(\text{"pay"}, pk_A, C'_A, pk_B, C'_B)$
Alice seals her opening to the enclave so Bob cannot read it
Bob sends the enclave v, the transaction, both signatures and both openings, then settles the result with the bank
Alice finishes with Bob's receipt or the public log

Chain of ownership
C_0 holds pk_0 $\xrightarrow{\text{signed by } sk_0}$ C_1 holds pk_1 $\xrightarrow{\text{signed by } sk_1}$ C_2
Every next key sits in a state the bank signed only after checking the previous key's signature: only the seed's owner can spend the chain, and no identity is involved

encrypted requests
signed result
Anonymous network
hides the sender

Bank's machine: relays encrypted requests; also sees attestations, results and timing

Enclave (attested code, run by the bank)
Runs the attested code correctly; may crash;
attested and registered by the bank at each boot;
keeps no payment state between requests

Keys, never stored
channel key: fresh at each attestation, decrypts one request, then erased
signing key (per boot): signs results enclave key

Checks a payment
E1 openings contain pk_A, pk_B; bank signatures valid enclave key
E2 new states open to $b_A - v$ and $b_B + v$ enclave key
E3 both owner signatures on m bank
E4 $0 < v \le b_A,\ b_B + v \le B_{\max}$ enclave key
in this order; one refusal for any failure

Sees: the openings of both states
Never sees: a user's seed or signing key

settle, refresh, enroll, read the log (anonymous)

Bank: issues the money, runs the enclave, signs new states
Follows the protocol and signs only valid results; analyses all it sees; may collude with users; keeps its records

Records
registered enclaves (attested signing keys)
spent serials
the stored answer to every settled request
its own state, used to issue and redeem
Not in it: users' balances, amounts of payments between users, identities

Checks a settlement
B1 recompute m; already settled: stored answer bank
B2 Alice's signature under pk_A, Bob's under pk_B bank
B3 result signed by a registered enclave enclave key
B4 two different, unspent serials; burn both, sign both new states, publish bank

Public transaction log (as in Platypus, but keeps every entry; anyone can read or mirror it)
per settled payment or refresh: the spent serials, two new commitments, the owners' and the enclave's signatures, the bank's signatures
used to finish a payment without a receipt and to recover a wallet from its seed; holds nothing the bank does not already see

What the bank learns

Operation	Sees	Never sees
Payment	two spent serials (and which one paid), two new commitments, three signatures, time	who, the amount, balances, the next serials
Refresh	one spent serial, one new commitment, time	who, the balance
Enroll	one new commitment, time	who, the first serial
Issue, redeem	its own side, the amount, the user's serial	the user's balance

Who guarantees what, and what happens if the enclave's memory is read (its code still runs correctly)

Property	Enforced by	Rests on	If the enclave's memory is read
Ownership: nobody spends another user's state	owner signature checked by the bank (B2)	bank	**holds**: signing keys never leave the devices; the enclave only sees signed transactions
A payment cannot be redirected or changed	both owners sign the exact new states (m)	bank	**holds**: a different payee or amount changes m; a replay finds the serial spent
No double spend	spent serials (B4)	bank	**holds**
Nobody can burn another user's state	burning a serial needs its owner's signature	bank	**holds**
Balance: no money is created	checked only by the enclave (E1, E2, E4), trusted through its signature (B3)	enclave key	**fails**: the attacker signs results for states that never existed, with any balance, and can redeem them; it still cannot take anyone's existing state
Privacy toward the bank	the enclave opens states, the bank sees only serials and commitments	enclave memory	**fails** for the requests read: amounts, balances, and links to each participant's previous and next payment; requests not read stay private

Integrity is not yet fully decoupled from the TEE: ownership is enforced by the bank, balance conservation still rests on the enclave's key.
Next step: let the bank check conservation without seeing values (zero-knowledge proofs as in Platypus, additive commitments with range proofs with or without TEE batching, or per-result attestation).